Handwritten Digit Recognition Using a Convolutional Neural Network

This project is the third step in a deliberate deep learning progression — from a single perceptron, to a fully connected artificial neural network, to a convolutional neural network — and the first to move from tabular data to images. A CNN was trained on the complete MNIST dataset of handwritten digits: 60,000 training images and 10,000 test images, each a 28×28 grayscale square labelled with its digit from 0 to 9. Pixel values were rescaled from 0–255 to 0–1, images reshaped to the four-dimensional format Keras expects, and labels one-hot encoded before training.

The architecture follows the classic CNN blueprint: a single Conv2D layer with 32 filters and a 3×3 kernel scans the image for local patterns such as edges and curves, a 2×2 max-pooling layer compresses the resulting feature maps, a flatten operation converts them into a one-dimensional vector, a 128-unit dense layer with ReLU activation combines the detected features, and a final 10-unit softmax layer outputs a probability distribution over the ten digit classes. The model was compiled with the Adam optimiser and categorical cross-entropy loss, then trained for 5 epochs at a batch size of 32, with 20% of the training data held out as a validation set throughout.

Training progressed rapidly: accuracy rose from 95.05% in epoch 1 to 99.45% by epoch 5, while validation accuracy climbed from 97.71% to a peak near 98.5% by epoch 4, settling at 98.45% in epoch 5. On the fully held-out test set of 10,000 unseen digits, the model achieved 98.56% accuracy with a loss of 0.0468 — correctly classifying roughly 986 of every 1,000 handwritten digits after only five epochs and a single convolutional layer, with no data augmentation.

The training curve is read honestly rather than simply celebrated: by epoch 5, training accuracy (99.45%) has pulled slightly ahead of validation accuracy (98.45%), a small generalisation gap indicating the model is just beginning to memorise training examples faster than it learns new general patterns. The gap is minor here, but it is the same signature that motivates dropout, data augmentation, and early stopping in production-grade CNNs — identified explicitly as this project's natural next extension.

The result demonstrates why convolution beats plain dense layers on images: a small filter shares its weights across the entire image instead of needing a unique parameter per pixel, giving translation-invariant pattern detection with far fewer parameters — the architectural leap behind modern computer vision.